

宁波大学
NINGBO UNIVERSITY

信息安全技术实验报告

[LSB 隐写实验]

姓名：	梁航锋
学号：	216001912
课程名称：	信息安全技术
所在学院：	信息科学与工程学院
专业班级：	2021 级计算机网络技术方向
任课教师：	严迪群

实验项目名称	LSB 隐写实验

一、 实验目的与要求：

1. 熟悉 CrypTool 的基本功能，掌握其中的 Graphical Editor 功能；
2. 掌握 LSB 等常见的图像隐写的基本原理与使用方式；
3. 对图像进行信息嵌入与取出，验证 LSB 隐写的正确性；
4. 使用 Python 等编程语言实现 LSB 隐写程序，深入理解隐写功能。

二、实验环境：

1. Windows11 Version 22H2
2. Python 3.8.12
2. CrypTool 2.1 软件

三、实验方法（原理、流程图）

实验过程中涉及到图像隐写技术，主要使用 LSB 进行本次实验。

3.1 图像隐写

图像隐写技术是一种将信息隐藏地嵌入到图像中的技术，同时不会引起人类视觉的察觉它可以用于隐蔽传输消息、数字水印、版权保护等目的。

常用的图像隐写技术包括最低有效位 LSB 隐写、图像文件格式隐写、使用密码学的隐写、基于直方图的可逆隐写、DCT 隐写、深度学习隐写等。

3.2 LSB 隐写

LSB 隐写是将要隐藏的信息嵌入到像素的最低有效位中，因为修改最低位对图像造成的变化较小，所以相对不容易被察觉。这是一种简单但是常用的隐写技术。

在数字图像中，每个像素通常由红色、绿色和蓝色三个颜色通道组成，每个通道的像素值通常用 8 位二进制数表示，即 0 到 255 之间的整数。在 LSB 隐写技术中，通常包含以下几个步骤：

1. 将要隐藏的信息转换成二进制形式；

2. 将每个二进制信息中的每一位依次嵌入到图像像素的最低位中。如果要隐藏的信息位是 0，则将像素的最低位设为偶数；如果要隐藏的信息位是 1，则将像素的最低位设为奇数。

3. 对于 RGB 图像，通常在三个颜色通道中分别嵌入信息，以提高隐写的容量。

4. 接收端收到带有隐藏信息的图像后，提取出像素的最低位，并将其还原成二进制形式，从而获取隐藏的信息。

对于最低位的选定也可以进行调整，为增加嵌入信息的容量，可以考虑选用最低的多位进行隐写。LSB 有一个明显的缺点，即容量有限且无法实现图像信息的可逆取出，即写入操作是不可逆的。

四、实验过程、步骤及内容

该部分内容给出本次实验的主要流程，具体的实验操作与现象在第五节中进行详细说明与分析。本次实验包含以下若干步骤：

（1） 使用 Cryptool 2 实现 LSB 的隐写实现，浅入信息需包含本人的学号或者姓名等信息；

（2） 使用 Python 代码实现简易的 LSB 隐写操作，并验证代码的正确性，在实现的代码上进行信息的嵌入和信息的提取等操作；

（3） 分析 LSB 隐写的实现流程，进一步理解其原理与优缺点；

（4） 学习其它常见的隐写技术，并适当进行拓展，例如学习掌握一些信息的可逆写入技术，并思考其可能的应用场景。

五、实验数据（现象）处理分析

该部分包含实验的主要操作过程与实验现象的记录。

5.1 使用 Cryptool 实现 LSB

根据实验要求，使用 Cryptool 软件中的 LSB 隐写模块进行信息的隐写，并对输出图像进行显示，同时对输出图像进行 LSB 的信息提取操作，整体的模块如图 5.1 所示。

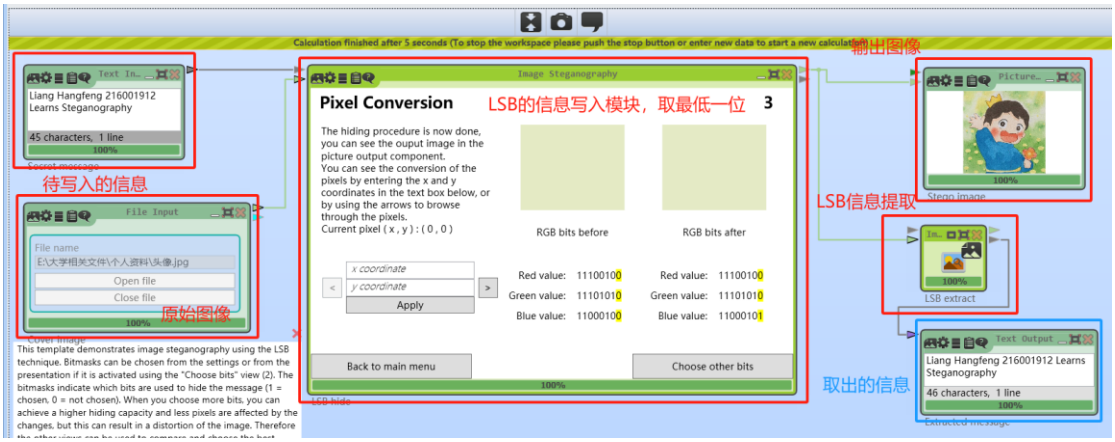


图 5.1 使用 Cryptool 实现 LSB

分析以上图可以看到，给定待写入的信息为“Liang Hangfeng 216001912 Learns Steganography”，共 45 个字符，每个字符对应的二进制位数为 8，则共有 360 位，对于原图像，像素为 690×690，为 RBG 图像，包含三个通道，因此选定写入到最后一位的话，显然可嵌入 690×690×3 位二进制信息，本次实验的写入信息符合该要求。

观察 LSB 嵌入后的图像，可以看到与原图几乎无差异，由于隐写只选用二进制的最后一位，对图像的色彩呈现影响不大。

将图像同步传输到 LSB 的信息提取模块，最终取出的信息也如图 5.2 所示，可以看到信息被无损取出。

5.2 使用 Python 实现信息的隐写

对于 LSB，其实现原理较为简单，整体步骤为：

1. 将待写入的信息转换为二进制；
2. 依次遍历图像的各个通道和各个像素点，例如按照长、宽、通道的顺序进行遍历，对每一个数据进行位操作，将当前的信息二进制位覆盖到数据的最后一个位；
3. 循环直至所有信息被嵌入。
4. 解密方面，按照以上顺序遍历图像，取出每一个图像像素数据的最后一位进行拼接，得到最终结果。
5. 以上解密会导致遍历整张图像，其中只有前面的信息是原嵌入的信息，因此考虑使用一些判定方式，例如特殊字符法。

根据以上步骤实现 Python 代码，进行模块化编程。

（1）信息嵌入函数实现

对于信息的嵌入函数，包含原视图像 image 和待嵌入的信息 message，其中 message 被添加了终止符号，用于判定信息到此结束，往后提取出来的信息为无

效信息。为方便排版，使用图像进行呈现，如图 5.2 所示。

```python
def encode_lsb(image, message):
    message = message + '&'
    binary_message = ''.join(format(ord(char), '08b') for char in message)  # 将消息转换为二进制
    height, width, channel = image.shape
    if len(binary_message) > height * width * channel:  # 检查消息是否过长
        raise ValueError("Message is too long to be embedded in the image")

    data_index = 0
    encoded_image = np.copy(image)
    for i in range(height):
        for j in range(width):
            for k in range(channel):  # 对RGB三个通道进行操作
                if data_index < len(binary_message):
                    # 使用位操作将信息嵌入到最后一位
                    encoded_image[i, j, k] = image[i, j, k] & 0b11111110 | int(binary_message[data_index])
                    data_index += 1
                else:
                    break
            if data_index >= len(binary_message):
                break
        if data_index >= len(binary_message):
            break
    return encoded_image
```

图 5.2 LSB 嵌入的 Python 实现

（2）信息提取函数实现

该函数传入参数为待提取信息的图像，遍历图像提取最低位并组合，注意结束字符的判定即可，函数实现如图 5.3 所示。

```python
def decode_lsb(image):
    binary_message = ''
    height, width, channel = image.shape
    for i in range(height):
        for j in range(width):
            for k in range(channel):  # 对RGB三个通道进行操作
                binary_message += str(image[i, j, k] & 1)  # 提取最低位信息

    message = ''
    for i in range(0, len(binary_message), 8):
        byte = binary_message[i:i + 8]
        if chr(int(byte, 2)) == '&':
            break
        message += chr(int(byte, 2))  # 将二进制转换为字符

    return message
```

图 5.3 LSB 提取的 Python 实现

（3）程序验证

调用两个函数，在主函数中分别进行图像读入、信息写入、信息写出、信息打印等操作，代码内容如图 5.4 所示。

```python
if __name__ == '__main__':
    # 读取图像
    image_path = './Test.jpg'
    image = cv2.imread(image_path)

    print(image.shape)
    show_image(image, 'original image')

    secret_message = 'Liang Hangfeng 216001912 Learns Steganography'

    emb_image = encode_lsb(image, secret_message)# 信息嵌入

    show_image(emb_image, 'emb_image')

    ex_message = decode_lsb(emb_image)# 信息提取

    print(ex_message)# 输出信息
```

图 5.4 LSB 的验证

输出的结果如图 5.5 所示，可以看到结果的与原始信息一致，由此判定信息的隐写操作与提取操作正确。

```
(690, 690, 3)
Liang Hangfeng 216001912 Learns Steganography

进程已结束,退出代码0
```

图 5.5 图像信息的提取

六、实验结论

本次实验基于 Cryptool2 软件和 Python 程序进行了 LSB 信息隐写技术的实现，并基于代码的编写进一步理解了 LSB 的实现原理。

对于 LSB，为最低有效位的信息隐写，选取图像中数据值的最后一个二进制数位进行信息写入，以实现在最小视觉干扰的基础上实现信息的写入，同时可以通过二进制数位的取出并组合实现信息的提取，这种方法的运行效率取决于信息的长度，与图像的规模无关，通过实现顺利的实现了信息的写入与取出。在 Python 代码的编写过程中，发现提取过程会遍历整张图片，从而提取出一些无效的信息，为了解决这种问题，考虑引入了结束字符来判定信息的结束。

但 LSB 存在一些缺点，例如信息的写入量取决于图像的大小，同时信息的写入对于图像来说本质上是一种不可逆的修改，在信息提取后图像无法完全恢复到原来的形态，因此产生了一系列图像的可逆隐写技术，例如基于直方图的图像隐写技术等。同时，如果图像收到的损坏，LSB 也无法进行检测，因此可以考虑结合以往学习的加密技术实现图像的损坏认证，例如将图像进行 hash 处理，并通过隐写将 hash 值进行嵌入，最后可以通过提取 hash 值来判定图像是否受损。

七、教师批阅意见：

成绩评定：

教师签字：

年　　月　　日

八、备注：